PROGRAM 6: Build an End-to-End RAG Pipeline

AIM

To develop an end-to-end RAG pipeline involving document loading, text chunking, embeddings, vector retrieval and answer generation.

ALGORITHM

1. Install the required libraries.
2. Prepare the source document.
3. Split the document into smaller chunks.
4. Generate embeddings for each chunk.
5. Store the embeddings in FAISS.
6. Convert the user's question into an embedding.
7. Retrieve the most relevant chunks.
8. Combine the retrieved chunks into context.
9. Generate an answer using the language model.
10. Display the answer and retrieved context.

CODE

Cell 1: Install libraries

```
!pip -q install sentence-transformers faiss-cpu transformers sentencepiece
```

Cell 2: Python program

```
import faiss
import numpy as np
import torch
from sentence_transformers import SentenceTransformer
from transformers import pipeline


# Step 1: Load models
embedder = SentenceTransformer("all-MiniLM-L6-v2")


generator = pipeline(
    "text2text-generation",
    model="google/flan-t5-small",
```

```
    device=0 if torch.cuda.is_available() else -1
)

# Step 2: Load source document
document = """
Artificial intelligence is a field of computer science.
Machine learning is a subset of artificial intelligence.
It enables systems to learn patterns from data.
Deep learning uses multi-layer neural networks.
Natural language processing deals with human language.
Computer vision helps computers interpret images.
"""

# Step 3: Text chunking
words = document.split()
chunk_size = 25
overlap = 5

chunks = []

for start in range(0, len(words), chunk_size - overlap):
    chunk = " ".join(words[start:start + chunk_size])
    if chunk:
        chunks.append(chunk)

print("Number of chunks:", len(chunks))

# Step 4: Generate chunk embeddings
vectors = embedder.encode(
    chunks,
```

```
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(vectors)

# Step 5: Build vector index
index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)

# Step 6: User question
question = "What does deep learning use?"

q_vector = embedder.encode(
    [question],
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(q_vector)

# Step 7: Retrieve relevant chunks
scores, ids = index.search(q_vector, min(2, len(chunks)))

context = "\n".join(chunks[i] for i in ids[0])

# Step 8: Generate answer
prompt = (
    "Answer the question using the provided context.\n"
    f"Context: {context}\n"
    f"Question: {question}\n"
    "Answer:"
```

```
)

answer = generator(
    prompt,
    max_new_tokens=80,
    do_sample=False
)[0]["generated_text"]

print("\nQuestion:", question)
print("\nRetrieved Context:\n", context)
print("\nAnswer:", answer)
```

INPUT

What does deep learning use?

OUTPUT

Number of chunks: [calculated number]

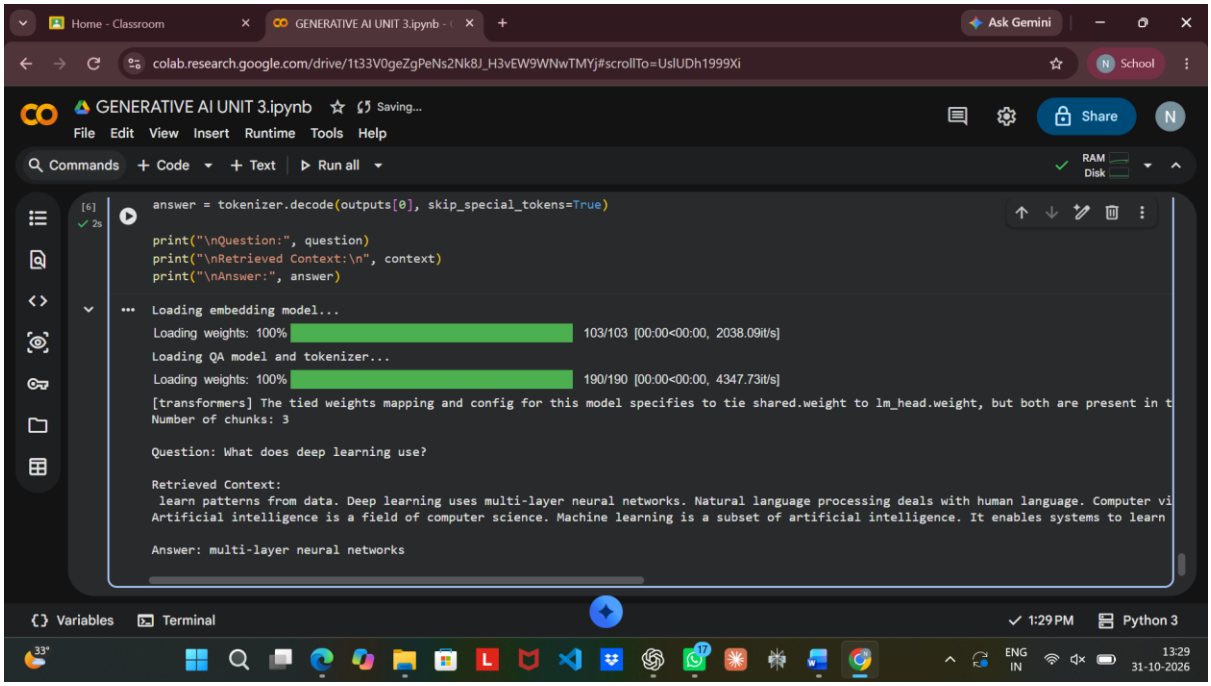
Question: What does deep learning use?

Retrieved Context:

[Relevant text chunks]

Answer:

Deep learning uses multi-layer neural networks.



RESULT

An end-to-end RAG pipeline was developed successfully, including document chunking, embedding generation, vector retrieval and answer generation.